

Periodic Multi-Agent Path Planning

Kazumi Kasaura, Ryo Yonetani, Mai Nishimura

OMRON SINIC X Corporation Hongo 5-24-5, Bunkyo-ku, Tokyo, Japan
kazumi.kasaura@sinicx.com, ryo.yonetani@sinicx.com, mai.nishimura@sinicx.com

Abstract

Multi-agent path planning (MAPP) is the problem of planning collision-free trajectories from start to goal locations for a team of agents. This work explores a relatively unexplored setting of MAPP where *streams* of agents have to go through the starts and goals with high throughput. We tackle this problem by formulating a new variant of MAPP called periodic MAPP in which the timing of agent appearances is periodic. The objective with periodic MAPP is to find a periodic plan, a set of collision-free trajectories that the agent streams can use repeatedly over periods, with periods that are as small as possible. To meet this objective, we propose a solution method that is based on constraint relaxation and optimization. We show that the periodic plans once found can be used for a more practical case in which agents in a stream can appear at random times. We confirm the effectiveness of our method compared with baseline methods in terms of throughput in several scenarios that abstract autonomous intersection management tasks.

Introduction

Multi-agent path planning (MAPP) refers to the problem of finding a set of collision-free trajectories from start to goal locations for a team of multiple agents. MAPP, specifically multi-agent pathfinding (MAPF) that searches for a solution on a given graph, is a fundamental problem in multi-agent systems (?).

We are particularly interested in the relatively unexplored problem of MAPP in which, rather than a single agent, a *stream of agents* enters each start location and leaves the environment upon reaching the goal. Instead of finding a set of feasible trajectories with a small total cost, we aim to improve the throughput for agent streams passing through the environment. Such settings would be beneficial for several practical applications, such as autonomous intersection management (AIM) (?) and automated warehouses (?).

Handling agent streams in such a problem setting poses a nontrivial technical challenge. As the throughput increases, the environment would be filled with a large number of agents, making it difficult to use optimal planning algorithms with limited scalability (*e.g.*, conflict-based search (?)). It is also not obvious if the high throughput can

be maintained with scalable planners that nevertheless have to determine agent trajectories adaptively in sequence (*e.g.*, prioritized planning (?)). Furthermore, finding collision-free trajectories in highly-crowded environments would require consideration of planning in the continuous space (*i.e.*, not grid maps) and with continuous time (*i.e.*, allowing agents to start and stop at an arbitrary timing in a continuous timeline). However, such continuous setups are generally challenging and there are few established approaches (??).

In this work, we start by formulating a bit simplified but new variant of MAPP called *periodic multi-agent path planning (periodic MAPP)* in which the timing of agent appearances is periodic. The objective with periodic MAPP is to find a *periodic plan*, *i.e.*, a set of collision-free trajectories that streams of agents can use repeatedly over periods. By finding such plans with periods that are as small as possible, we are able to improve the throughput of agent streams. Importantly, periodic plans once found can be easily used for solving a more practical problem called online MAPP, a variant of online MAPF (?) in which a stream of agents can appear at random times but can also wait until the subsequent agents enter the environment.

We develop a constraint-relaxation and optimization method as a solution method to periodic MAPP. With this method, we first generate an initial periodic plan under relaxed constraints about the physical size of agents with an arbitrarily large period they appear. We then solve a continuous optimization problem to improve the initial plan such that the agent size matches the original one and the period becomes as small as possible. Therefore, our method can find a collision-free and repeatable plan while minimizing the period. We provide insights into the topological aspect of solutions obtained with the proposed method.

We evaluated the effectiveness of our method on several scenarios of abstracting AIM tasks, in which the goal is to move vehicles appearing at intersections to the other side without collision. Unlike existing methods that require planning or re-planning for every appearance of a new vehicle, the proposed method using periodic plans does not necessitate communications with other vehicles to retrieve their current locations or to update their trajectories. Nevertheless, the solutions derived from the proposed method are comparably good or sometimes even better in terms of the throughput, compared with those from baseline

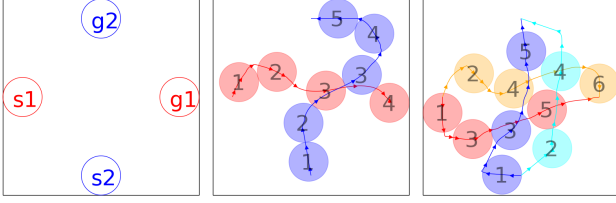


Figure 1: Example of periodic MAPP problem with $N = 2$ (left) and periodic plans with $M = 1$ (middle) and $M = 2$ (right). Solid lines show trajectories. Numbered circles indicate where agents are at each elapsed period after their appearance.

methods that combine online MAPP algorithms (?) and MAPP algorithms for continuous spaces and times (???)

Periodic MAPP

Overview. We consider a two-dimensional (2D) environment with several pairs of start and goal locations. For each start location, a stream of agents appears periodically with a user-defined *period* (i.e., time interval). Each agent must move to its goal while avoiding collisions with the borders of the environment and other agents and disappear from the environment upon reaching the goal. We assume that there exists a certain *cycle*, the number of periods within which we can find a *periodic plan* (i.e., a set of collision-free trajectories that can be used periodically over cycles). Therefore, a periodic plan may span multiple periods as collision-free trajectories for agents appearing at the same locations that are not necessarily the same across periods (see agents shown in red/orange or those in blue/cyan in Fig. 1). Informally, the objective with periodic MAPP is to find such periodic plans for a given cycle with periods that are as small as possible. In other words, we wish to produce a high throughput plan that enables us to move agents from their respective start to the goal even if they appear in rapid succession. Note that, if the non-periodic version of a given problem instance (i.e., standard MAPP with a single agent appearing from each start) has a solution, the problem instance has a periodic plan for any cycle with the period given by the arrival time of the last agent. For simplicity, we assume that all agents have bodies modeled by circles with the same fixed radius and follow a simple kinodynamic constraint in which the velocity cannot exceed a certain maximum limit.

Problem instances. Formally, we model a problem instance of periodic MAPP by a tuple $(\mathcal{E}, \{(s_1, g_1), \dots, (s_N, g_N)\}, r, v_{\max})$, where $\mathcal{E} \subseteq \mathbb{R}^2$ is a set of valid states in the 2D environment. The set $\{(s_1, g_1), \dots, (s_N, g_N)\}$, where $s_n, g_n \in \mathcal{E}$, describes N pairs of start and goal locations for agent streams. The variables r and $v_{\max}$ are the radius and maximum velocity of each agent, respectively.

Periodic plans. We refer to a *period* as $\tau \in \mathbb{R}_+$, a time interval with which a new set of agents can appear at respective start locations $s_1, \dots, s_N$. While denoting *cycle* as $M \in \mathbb{N}_+$, we describe a *periodic plan with M periods* by

a set of $M \times N$ trajectories $\Gamma_M = (\gamma_{n,m} : [0, T_{n,m}] \rightarrow \mathcal{E})_{1 \leq n \leq N, 0 \leq m < M}$. The periodic plan should satisfy the following conditions:

- (Start and goal locations) For all $1 \leq n \leq N$, $0 \leq m < M$, $\gamma_{n,m}(0) = s_n$, and $\gamma_{n,m}(T_{n,m}) = g_n$.
- (Maximum velocity) For all $1 \leq n \leq N$, $0 \leq m < M$, and $t \in [0, T_{n,m}]$, the velocity of agents satisfies

$$\left| \frac{d\gamma_{n,m}}{dt}(t) \right| \leq v_{\max}. \quad (1)$$

- (Clearance from boundaries) Let $\text{dist}_{\mathcal{E}}(x)$ be the distance of $x \in \mathcal{E}$ from the boundary of $\mathcal{E}$. Then, for all $1 \leq n \leq N$, $0 \leq m < M$, and $t \in [0, T_{n,m}]$,

$$\text{dist}_{\mathcal{E}}(\gamma_{n,m}(t)) \geq r. \quad (2)$$

- (Collision-free)¹ For all $1 \leq n, n' \leq N$, $0 \leq m, m' < M$, and $t \in [0, T_{n,m}]$, $t' \in [0, T_{n',m'}]$ such that $(m - m')\tau + (t - t') \in M\tau\mathbb{Z}$ and $(n, m, t) \neq (n', m', t')$,

$$|\gamma_{n,m}(t) - \gamma_{n',m'}(t')| \geq 2r. \quad (3)$$

Objective of periodic MAPP. Given a problem instance $(\mathcal{E}, \{(s_1, g_1), \dots, (s_N, g_N)\}, r, v_{\max})$ and a cycle M , our objective is to find periodic plans Γ_M with periods τ that are as small as possible.

Solution Method

In this section, we explain the proposed solution method for producing periodic plans for periodic MAPP. Specifically, we use a two-step approach that first derives initial solution trajectories for a relaxed problem that ignores the constraints about r and the objective for τ . We then optimize them by solving a continuous optimization problem to satisfy all the original conditions and improve the solution quality with respect to τ . This is a reasonable approach to derive solution trajectories in a continuous space and time setup while aiming to minimize the continuous period value that affects the solution.

Trajectory Representation

We represent each trajectory $\gamma_{n,m}$ by a sequence of $K + 1$ timed locations with a timestep $\Delta t_{n,m}$, i.e., $((x_{n,m,0}, 0), (x_{n,m,1}, \Delta t_{n,m}), \dots, (x_{n,m,K}, K\Delta t_{n,m}))$ where $x_{n,m,k} \in \mathcal{E}$, $x_{n,m,0} = s_n$, $x_{n,m,K} = g_n$, and $K\Delta t_{n,m} = T_{n,m}$. Agents are assumed to move between two locations $x_{n,m,k}, x_{n,m,k+1}$ in a straight line with constant velocity. This discretized representation should also satisfy the maximum velocity condition in Eq. (1) that is rewritten as

$$v_{n,m,k} := \left| \frac{x_{n,m,k+1} - x_{n,m,k}}{\Delta t_{n,m}} \right| \leq v_{\max}, \quad (4)$$

¹The agent appearing at s_n at time $(aM + m)\tau$ follows the trajectory $\gamma_{n,m}$ for any $a \in \mathbb{Z}$. Since there exists an agent at $\gamma_{n,m}(t)$ when the time is $\dots, (-2M + m)\tau + t, (-M + m)\tau + t, m\tau + t, (M + m)\tau + t, (2M + m)\tau + t, \dots$, there exist two agents at $\gamma_{n,m}(t)$ and $\gamma_{n',m'}(t')$ at the same time if $(m - m')\tau + (t - t') \in M\tau\mathbb{Z}$, except when $(n, m, t) = (n', m', t')$. To avoid a collision between them, their distance must be at least $2r$.

and that for the clearance from boundaries in Eq. (2):

$$\text{dist}_{\mathcal{E}}(x_{n,m,k}) \geq r. \quad (5)$$

Note that satisfying the collision-free condition in Eq. (3) is a bit non-trivial. Let us define

$$r_q(t) := t - \left\lfloor \frac{t}{q} \right\rfloor q, \quad (6)$$

and

$$\begin{aligned} C := \{ & (n, m, k, n', m', k') | \\ & 1 \leq n, n' \leq N, 0 \leq m, m' < M, 0 \leq k, k' < K, \\ & 0 \leq r_{M\tau}((m - m')\tau + k\Delta t_{n,m} - k'\Delta t_{n',m'}) < \Delta t_{n',m'}, \\ & (n, m, k) \neq (n', m', k') \}. \end{aligned} \quad (7)$$

Then, the collision-free condition is rewritten as, for all $(n, m, k, n', m', k') \in C$,

$$\begin{aligned} d_{n,m,k,n',m',k'} := & |x_{n,m,k} - ((1 - \alpha)x_{n',m',k'} + \alpha x_{n',m',k'+1})| \\ & \geq 2r, \end{aligned} \quad (8)$$

where

$$\alpha = \frac{r_{M\tau}((m - m')\tau + k\Delta t_{n,m} - k'\Delta t_{n',m'})}{\Delta t_{n',m'}}. \quad (9)$$

Note that by setting $t = k\Delta t_{n,m}$, $t' = (k' + \alpha)\Delta t_{n',m'}$, the inequality in Eq. (8) reduces to the original one of Eq. (3).

Optimization

Initial periodic plans. We first create an initial periodic plan while setting r smaller than that of the original condition and τ large enough. This makes it easy to find feasible trajectories that satisfy the above conditions. Concrete algorithms used to produce such plans depend on task setups, which we present in Appendix B.

Objective. Given an initial periodic plan for $\Gamma_M = (\gamma_{n,m})_{1 \leq n \leq N, 0 \leq m < M}$, we optimize it with respect to r , τ , and each trajectory $\gamma_{n,m}$ to satisfy the original conditions. We denote the original agent radius as r_0 . By imposing the cost to violate the original conditions, solving periodic MAPP reduces to a continuous optimization problem with the following objective:

$$V(\tau, r, \Gamma_M) := \left(\tau - \frac{2r}{v_{\max}} \right)^2 + \frac{\sigma_t}{K} \sum_{n,m,k} v_{n,m,k}^2 + c(\tau, r, \Gamma_M), \quad (10)$$

$$\begin{aligned} c(\tau, r, \Gamma_M) := & \sigma_r(r - r_0)^2 \\ & + \frac{\sigma_v}{K} \sum_{n,m,k} (\max\{0, v_{n,m,k} - v_{\max}\})^2 \\ & + \frac{\sigma_o}{K} \sum_{n,m,k} (\max\{0, \text{dist}_{\mathcal{E}}(x_{n,m,k})^{-1} - r^{-1}\})^2 \\ & + \frac{\sigma_c}{K} \sum_C \left(\max\left\{0, d_{n,m,k,n',m',k'}^{-1} - (2r)^{-1}\right\} \right)^2, \end{aligned} \quad (11)$$

where σ_t , σ_r , σ_v , σ_o and σ_c are constants. With this objective, we aim to decrease τ to the minimum $2r/v_{\max}$ where two agents are adjacent to each other. We also impose costs of trajectories defined by the sums of the squares of velocity to prevent vanishing of the gradients on the trajectories.

Optimization method. We solve this optimization problem by using the Levenberg-Marquardt algorithm (??). To force solutions to strictly satisfy the original conditions, we make the constants σ_r , σ_v , σ_p , σ_c gradually increase to become large enough during the optimization. We also gradually decrease σ_t up to zero because the corresponding velocity term is necessary only for preventing vanishing gradients and is not included in the original conditions.

Topological Remark

The quality of final solutions is dependent on initial periodic plans, while some initial plans will result in the same optimization results. One considerable feature of solutions is their equivalent classes with respect to continuous deformation, including optimizations, from topological perspectives.

To analyze this, we introduce an additional constraint in which no agents can pass through any start and goal locations, including those of themselves, instead of considering the conditions of the velocity and size of agents. This is necessary to ensure sufficiently different plans to be distinct enough in terms of their homotopy class. Let:

$$\mathcal{C} := (\mathcal{E} \setminus \{s_1, g_1, \dots, s_N, g_N\}) \times \mathbb{R}/\mathbb{Z}. \quad (12)$$

A trajectory $\gamma_{n,m}$ can be considered an embedding $\tilde{\gamma}_{n,m}$ of an open interval $(0, 1)$ to $\mathcal{C}$:

$$(0, 1) \ni \alpha \mapsto \left(\gamma_{n,m}(tT_{n,m}), \overline{\left(\frac{m\tau + \alpha T_{n,m}}{M\tau} \right)} \right) \in \mathcal{C}. \quad (13)$$

where the overline means the equivalent class. Then, a periodic plan $(\gamma_{n,m})_{1 \leq n \leq N, 0 \leq m < M}$ can be considered an embedding $(\tilde{\gamma}_{n,m})_{1 \leq n \leq N, 0 \leq m < M}$ of the disjoint union of $N \times M$ open intervals $(0, 1)$ to $\mathcal{C}$, satisfying the following conditions for any $1 \leq n \leq N$ and $0 \leq m < M$:

- $\lim_{\alpha \rightarrow 0} \tilde{\gamma}_{n,m}(\alpha) = (s_n, \overline{m/M})$ and $\lim_{\alpha \rightarrow 1} \tilde{\gamma}_{n,m}(\alpha) = (g_n, \overline{\mathcal{A}})$ for some $\mathcal{A} \in \mathbb{R}/\mathbb{Z}$.
- The second component of $\tilde{\gamma}_{n,m}(\alpha)$ is locally strictly increasing with respect to α .

Note that the collision-free condition can be interpreted as injectivity.

Now, a set of plans that are equivalent with respect to continuous deformations is the set of homotopy classes of the embeddings that satisfy the above conditions, and the following proposition holds.

Proposition 1 *When $\mathcal{E}$ is open and connected, the above set is independent of the positions of $s_1, g_1, \dots, s_N, g_N$.*

For the proof, see Appendix A.

Application to Online MAPP

While it is assumed with periodic MAPP that the appearance timing of agents in a stream is periodic, the periodic plans once found can be used for solving a more practical problem called online MAPP, where agents appear at *random times*.

Online MAPP Problem

While sharing certain settings with periodic MAPP, the problem of online MAPP can be viewed as a variant of online MAPF (?) with the following features. Like periodic MAPP, there are several pairs of starts and goals in the environment through which streams of agents have to pass while avoiding collisions. Following online MAPF, we assume that agents can appear at random times but are also allowed to wait at their start locations in a finite or infinite queue until the subsequent agents enter the environment. This assumption is similar to the concept of ‘garage’ (?) and realistic for practical applications such as AIM.

Proposed Method

Adopting periodic plans to agents with random timing of appearances is straightforward. We first divide a timeline into periods of the interval τ obtained with the periodic plan. We then allow at most one agent for each period to enter each start location and follow the corresponding trajectory. Therefore, agents can avoid collisions with other agents in the same stream.

Formally, by using the notations of periodic MAPP introduced earlier, we say that an agent is assigned to the n -th trajectory in the a -th period when it waits until time $a\tau$ and moves along $\gamma_{n,a-\lfloor a/M \rfloor M}$. Let us denote as t , the time of appearance of a new agent at the start position s_n . Let $a := \lceil t/\tau \rceil$. If the n -th trajectory in the a -th period has not yet been assigned by any agent, the new agent will follow that trajectory. Otherwise, the agent will wait to follow the next trajectory in the $a' + 1$ -th period, where a' is the order of the period assigned to the last agent that appeared at s_n before t if the queue has a room. Note that if the length of the queue is finite and the number of currently waiting agents exceeds its limit, the planning for the new agent is considered a failure.

Queueing Theoretical Analysis

We theoretically analyze the waiting times of agents for the proposed method.

Assumptions. We assume that agents are managed by queues of infinite length. We also model time intervals of agent appearances as $c + \alpha$, where c is a constant² and α is a random variable drawn from the exponential distribution with a rate parameter λ . We also assume that all agents wait for (the maximum) time τ until they start moving. Note that this assumption corresponds to the deterministic service time in terms of the queueing theory (?) and is more conservative than in actuality.

Waiting time analysis. For theoretical analysis, we remove a constant term from the above model by subtracting c from time intervals between arrivals and service time temporarily. This operation does not change the number of agents in a queue but reduces waiting times for all the agents. The resulting model then reduces to the M/D/1 queue in terms of the queueing theory (?). By denoting the rate of

arrivals as λ and the service time as $D := \tau - c$, and when $\rho = D\lambda < 1$, the average waiting time is given as

$$W' = D + \frac{\rho}{2(1-\rho)}D. \quad (14)$$

Moreover, the probability that the waiting time exceeds a limit t decreases exponentially with respect to t (?). By again considering c we subtracted earlier, the average time now becomes

$$W = W' + c = \tau + \frac{\lambda(\tau - c)^2}{2(1 - \lambda(\tau - c))}. \quad (15)$$

Experiments

To evaluate the effectiveness of our method for first solving periodic MAPP then using periodic plans for online MAPP problems, we focus on scenarios of abstracting AIM tasks (?).

Experimental Setups

Environments. Our AIM scenario involves a single intersection with several entrances and exits. Figure 2 shows the six environments used in the experiments. Each environment abstracts one of the typical situations of intersections with different sizes. The circles with letters ‘s’ and ‘g’ are the start and goal locations, respectively. Specifically, environment (a) is a crossing of two one-way roads, while environment (b) is a crossing of one-way and two-way roads. Environments (c) and (d) are modeling the crossing of two two-way roads of different sizes. Finally, environments (e) and (f) model the T-junction with different sizes. We also assume that each start location is equipped with a ‘corridor’, as illustrated in Fig. 4, which we can use as a queue (with unlimited or limited capacity) to keep agents waiting until entering the intersection. Agents (*i.e.*, vehicles) are modeled by a circle and follow a simple holonomic kinodynamics model that enable them to move in any direction under a given maximum velocity. Trajectories for a new agent must be planned immediately once that agent appears in the environment. Nevertheless, it is possible to replan trajectories for some agents that have already been moving in the environment to take into account the new agent.

Parameters. We evaluated two different configurations for the queues: unlimited or limited capacity with five agents at most. The time interval between agent appearances is modeled as $1.0 + \alpha$, where α follows the exponential distribution with a rate parameter λ . We sampled appearance times until the last time reached 1000 for infinite queues and 100 for finite queues. For each environment and each $\lambda \in [0.25, 0.5, 0.75, \dots, 2.5]$, we generated 10 different problem instances. Throughout the experiment, r and $v_{\max}$ were respectively fixed to 0.5 and 1.0.

Evaluation metrics. The quality of plans is measured by the following two metrics.

- **Throughput** measured as the number of agents entering the environment in unit time.

²We add c to account for the time margin needed to avoid collisions between agents.

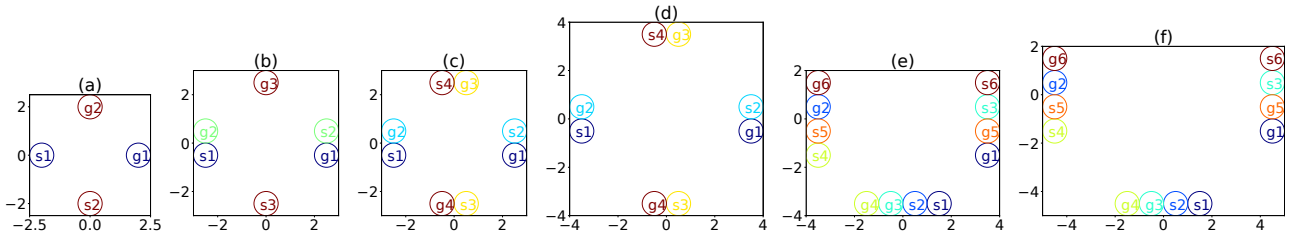


Figure 2: Environments that abstract AIM tasks. Letters ‘s’ and ‘g’ indicate start and goal locations, respectively.

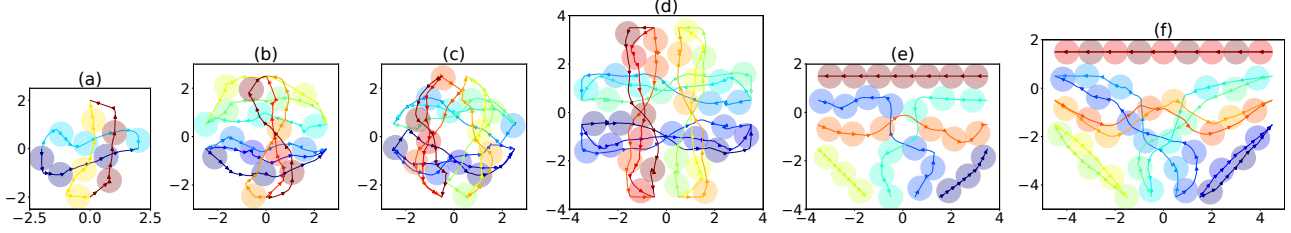


Figure 3: Optimized periodic plans for each environment with best M . Best viewed in videos in the Appendix.

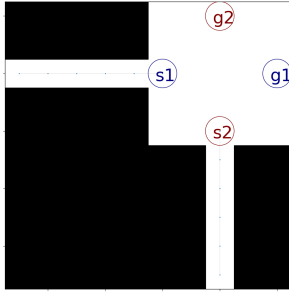


Figure 4: Environment (a) with corridors used as queues for five agents.

- **Average delay** calculated as the incremental travel time compared with the shortest possible trajectory averaged over agents. Note that this metric includes the time for the agent waiting in a queue.

Initial Periodic Plans

With periodic MAPP, we generated and evaluated three periodic plans for each problem instance with cycles $M \in \{1, 2, 3\}$. We constructed the plans by carefully designing the order of passing at intersection points such that, after M agents of one direction pass at the intersection, M agents of another direction pass alternately.

Formally, the initial plan for M was created on the basis of the following rules:

- Let π_n be the shortest path connecting s_n and g_n . Each agent appearing at s_n follows π_n , while adjusting its velocity to satisfy the next condition.
- Let two paths π_n and $\pi_{n'}$ intersect at point p . We assume that the length of the part of π_n from s_n to p is shorter than that of the part of $\pi_{n'}$ from $s_{n'}$ to p (and assume

$i < i'$ if they are equal). Then, for any $a \in \mathbb{Z}$, M agents appearing at s_n at time $aM\tau, (aM + 1)\tau, \dots, (aM + M - 1)\tau$ must pass p at the time between times when agents appearing at $s_{n'}$ at $(aM - 1)\tau$ and at $aM\tau$ pass p . The differences between times when two agents pass are set to be not smaller than 1.0 to prevent optimization failures.

Note that such initial plans can always be constructed as long as the initial τ is taken to be large and r is small enough. See Appendix B for details of the generation algorithm.

Baseline Methods

Because the same problem setup (*i.e.*, improving throughput of agent streams for MAPP in continuous space and time) has not been explicitly addressed, we developed two baseline methods called *first-come and first-serve (FCFS)* and *snapshot optimal (SO)*, which combine general strategies of online MAPF (?) and the state-of-the-art MAPP algorithms in continuous space and time (??). For both baselines, we first used probabilistic roadmap with the fixed number of neighbors (k-PRM) (?) as a standard approach to approximate the continuous space into a roadmap and find collision-free paths on the constructed roadmap. Implementation details are presented in Appendix C.

- **FCFS**: This baseline incrementally plans a collision-free trajectory for every new agent that appears while regarding trajectories of other agents already present in the environment as space-time obstacles. This is a natural application of prioritized planning (?) for the “replan single” strategy introduced in the context of online MAPF (?). Specifically, we used prioritized safe-interval path planning (?) to handle continuous space and time.
- **SO**: By contrast, this baseline uses the “replan all” strategy (?) and finds the optimal solution using continuous conflict-based search (?) for all the agents presenting in the environment each time a new agent appears.

M	Environment					
	(a)	(b)	(c)	(d)	(e)	(f)
1	2.03	2.08	2.45	2.44	2.34	2.13
2	1.27	1.54	1.87	1.86	3.02	1.86
3	1.56	1.45	1.86	1.63	3.04	3.54

Table 1: Periods for optimized periodic plans with different cycle M

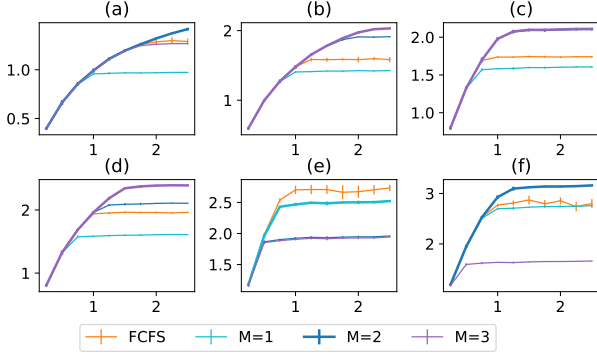


Figure 5: Throughput with respect to λ for *infinite* queues (bars represent standard derivations)

Results

Optimization results. Table 1 shows the periods of optimized plans for cycle $M = 1, 2, 3$ and Fig. 3 illustrates the optimized periodic plans with the best choice of M . We would like to emphasize that this optimization is required only once for each combination of environments and M . Unlike the other baselines, our method can be used without requiring agents to communicate with other agents to retrieve their current locations or replanning to adjust their trajectories for each appearance of new agents.

Case I: Infinite queues. Figures 5 and 6 respectively show the changes in throughput and average delays with respect to λ when the capacity of queues is infinite. With the exception of the environment (e), our method, especially for $M = 2$ or $M = 3$, consistently outperformed FCFS as λ became higher. Note that it was not possible to use SO because the number of agents present in the environment could quickly become very large. We observed that the limited performance of the proposed method for (e) is possibly due to the difficulty of finding enough room for periodic intersections in the environment.

Case II: Finite queues. We set the length of queues to be five. Note that planning for a new agent is regarded as failed if the number of agents waiting to enter their starts exceeded the length of queues or when the planning time exceeds a predefined limit (1.0 s). Agents resulting in such planning failures are discarded and degrade throughput. Figures 7 and 8 respectively show the throughput and average delays changing with respect to λ . Regarding throughput,

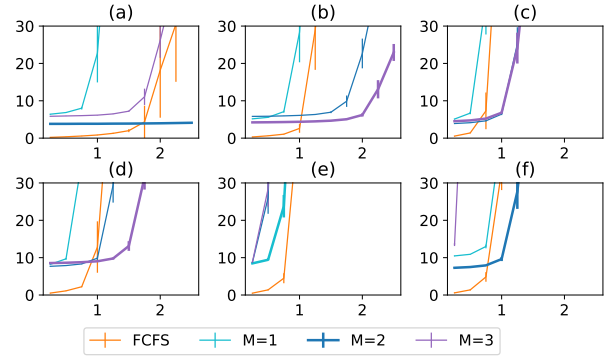


Figure 6: Average delay with respect to λ for *infinite* queues (bars represent standard derivations)

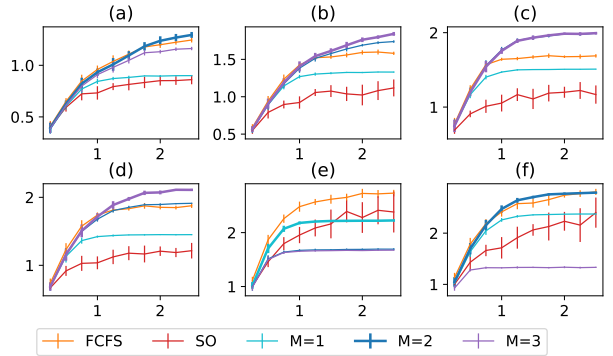


Figure 7: Throughput with respect to λ for *finite* queues (bars represent standard derivations)

our method for $M = 2$ and $M = 3$ performed better than FCFS and clearly outperformed SO, except for environment (e). The performance of SO degraded mainly due to the planning failures. By contrast, our method showed limited performances in terms of average delays, indicating the limitation of our method that trajectories obtained using periodic plans would be relatively redundant compared with those from FCFS and SO.

Related Work

MAPP and its variants. MAPP has historically been studied in the fields of artificial intelligence and robotics. Many studies have considered planning particularly in discrete spaces such as grid maps, which are often referred to as MAPF (e.g., ??, and see ? for an extensive survey.) There are a number of variants of MAPF problems, such as Anonymous MAPF (i.e., no correspondences between starts and goals) (??), MAPF with kinodynamic constraints (??), and online MAPF (???), also known as lifelong MAPF. As summarized by ?, online MAPF is an online version of MAPF in which a team of agents is asked to solve a stream of tasks. Agents are assigned a new task whenever they appear in the environment (?) or upon reaching their goal (?). In contrast, periodic MAPP and our version of online MAPP are different in that streams of agents are asked to solve a certain task,

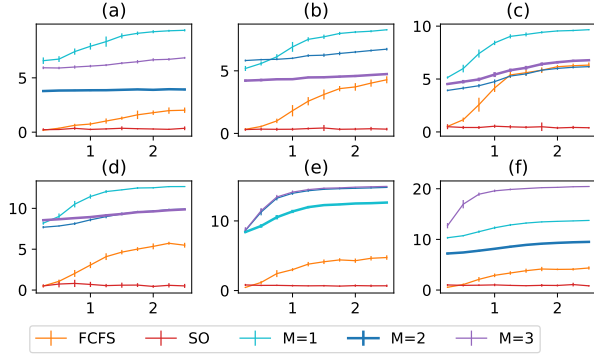


Figure 8: Average delay with respect to λ for *finite* queues, (bars represent standard derivations)

with the unique objective that aims for high throughput. We introduced a challenging problem setup that solves online MAPP in the continuous space and with continuous time. Studies on such continuous setups have significantly been limited compared with discrete cases (??????).

Application to AIM. As reviewed by ??, AIM is a common application of online MAPF. Typical approaches include the first-come and first-serve for repeatedly determining trajectories for every new agent (?) and the application of optimal solvers for a set of all agents present at the moment (?), which we compared in our experiments. For a given trajectory (or simply lane), there are studies that used deep reinforcement learning to achieve an optimal policy for vehicle acceleration control (??).

Optimization for MAPP. Similar to this work, other studies use continuous optimization of trajectories (*i.e.*, trajectory deformation (?)), especially for single-agent cases. For example, the idea of avoiding collisions on the basis of the optimization has existed for a long time (?). Another widely used motivation for optimization is to take into account kinodynamic constraints, *e.g.*, that in ? for a single-agent case and that in ? for a multi-agent case. Our study is unique in that we optimized trajectories as well as the period for their repeated use.

Other related work. Finally, our study has several more connections to other prior works. Solving path planning while relaxing constraints is a technique for single-agent cases (???). The homotopical aspect of path planning has been studied by ? for single-agent and by ? for multi-agent cases. However, these studies are not directly applicable to our problem setup due to the difficulty of considering collisions for agents appearing in a periodic fashion.

Conclusion

We presented a new variant of MAPP called periodic MAPP in which a stream of agents can appear periodically at each start location and leave the environment once they arrive at the goal. We also proposed a solution method of periodic MAPP that generates a periodic plan, *i.e.*, a set of collision-free trajectories that can be used repeatedly over periods

while maintaining high throughput. We showed that the periodic plans can further be used for solving the online MAPP problem in which agents in each stream appear at a random time, and demonstrated its effectiveness on scenarios of abstracting AIM tasks.

Currently, our formulation of periodic MAPP as well as our solution method can cover only situations in which agents follow a simple kinodynamics model that takes into account only maximum velocity. Promising future work is to address more realistic kinodynamics of wheeled robots or drones. This would require extending an optimization technique for solution methods such as ones used to plan trajectories for swarms (?). Another interesting direction from an application perspective is to tackle a more challenging AIM task in which human-driven vehicles also exist. In such a case, it will be important to combine the proposed method with state-of-the-art AIM methods that can reactively control acceleration on a given trajectory (??).

A: Proof of Proposition 1

Let $s'_1, g'_1, \dots, s'_N, g'_N$ be another collection of start and goal locations. It is enough to show that there exists a homeomorphism $F : \mathcal{E} \rightarrow \mathcal{E}$ such that $F(s_n) = s'_n$ and $F(g_n) = g'_n$ for all $1 \leq n \leq N$. This is proven by ?.

B: Generation Algorithm of Initial Plans

First, we set $r = 0$ to relax the clearance from boundaries and collision-free constraints. Let $s_n = p_{n,0}, p_{n,1}, \dots, p_{n,K_n} = g_n$ be start and goal locations and all intersection points with other paths on π_n ordered from s_n to g_n . Also, let $t_{n,m,k}$ be the traveling time from s_n to $p_{n,k}$ in the initial plan, and $L_{n,k}$ be the minimum traveling time from $p_{n,k}$ to $p_{n,k+1}$. Then, the following condition must hold:

$$t_{n,m,0} = 0, \quad (16)$$

$$t_{n,m,k} + L_{n,k} \leq t_{n,m,k+1}, \quad (17)$$

We also consider constraints of the forms “the m -th agent appeared s_n must pass through $p_{n,k} = p_{n',k'}$ earlier than m' -th agent appeared s_n ”, where $0 \leq m, m' < M$, which is written by:

$$m\tau + t_{n,m,k} < m'\tau + t_{n',m',k'}. \quad (18)$$

We describe these constraints using tuple $C_1 := (n, m, k, n', m', k')$. Likewise, there exist constraints of the forms “the $(M-1)$ -th agent appeared s_n must pass through $p_{n,k} = p_{n',k'}$ earlier than M -th agent appeared $s_{n'}$ ”. They are given as follows:

$$(M-1)\tau + t_{n,M-1,k} < M\tau + t_{n',0,k'}, \quad (19)$$

which we describe them using tuple $C_2 := (n, k, n', k')$.

Moreover, we set a redundancy parameter $R = 1.0$ and make these constraints stricter:

$$m\tau + t_{n,m,k} + R \leq m'\tau + t_{n',m',k'}. \quad (20)$$

$$(M-1)\tau + t_{n,M-1,k} + R \leq M\tau + t_{n',0,k'}. \quad (21)$$

After the values of $t_{n,m,k}$ satisfying the above conditions are determined, the initial value of $\gamma_{n,m}$ can be constructed by connecting $p_{n,k}$ and $p_{n,k+1}$ with constant velocity motions.

Now, we construct a directed graph such that its vertices are tuples (n, m, k) and its edges connect from (n, m, k) to $(n, m, k + 1)$ or from (n, m, k) to (n', m', k') for $(n, m, k, n', m', k') \in C$. Then, for the constraints given in this paper, the graph is acyclic and the indegrees of vertices of the form $(n, m, 0)$ are zero. So we can compute the minimum values of $t_{n,m,k}$ satisfying (16), (17), and (20) by dynamic programming after the value of τ is determined.

Since the value of τ is not determined yet, we write $t_{n,m,k} = a_{n,m,k}\tau + b_{n,m,k}$ and define the order of pairs of $(a, b) \in \mathbb{Z} \times \mathbb{R}$ by the lexicographical order. Then, we can compute the minimum values of $a_{n,m,k}, b_{n,m,k}$ by dynamic programming satisfying the following conditions:

$$(a_{n,m,0}, b_{n,m,0}) = (0, 0), \quad (22)$$

$$(a_{n,m,k}, b_{n,m,k} + L_{n,m}) \leq (a_{n,m,k+1}, b_{n,m,k+1}), \quad (23)$$

$$(a_{n,m,k} + m, b_{n,m,k} + R) \leq (a_{n',m',k'} + m', b_{n',m',k'}) \quad (24)$$

for all $(n, m, k, n', m', k') \in C_1$.

We determine the values of $a_{n,m,k}, b_{n,m,k}$ as the minimum ones. Furthermore, we can prove that these values hold the following inequality by the forms of conditions:

$$a_{n,m,k} + m \leq M - 1. \quad (25)$$

Thus, when τ is large enough, $t_{n,m,k} = a_{n,m,k}\tau + b_{n,m,k}$ while satisfying the conditions (16), (17), (20), and (21). We determine the value of τ as the minimum one of such values.

C: Implementation Details

All the methods are implemented in C++ and evaluated with Intel Core i9-9900K CPU and 32GB RAM.

Optimization method. For optimization, we use Levenberg-Marquardt (LM) Algorithm implemented in g²o (?). Parameters for the optimization are set as the default values of the library.

We set $\sigma_r = \sigma_v = \sigma_o = \sigma_c = 10^4$ and $\sigma_t = 1$ as the initial values. Since the set C in Eq.(7) of the main paper may change depending on the value of p or $T_{i,j}$, we have to reset the explicit constraints repeatedly during optimization. The number of iteration of LM algorithm and intervals of resetting constraints are as follows:

1. The first 500 iterations: we recalculate C after each iteration.
2. The next 39500 iterations: we recalculate C for 10 iterations.
3. The following 1000 iterations: we recalculate C and increase $\sigma_r, \sigma_v, \sigma_o$, and σ_c by multiplying 1.01 and decrease σ_t by dividing 1.01 after each iteration.
4. The remaining iterations: we recalculate C after each iteration. This phase lasts until the plan converges, more precisely, until the difference of costs for 100 iterations goes below 10^{-6} .

Table 2 reports the number of iterations until the optimization converges and required times in minutes.

M	Environment					
	(a)	(b)	(c)	(d)	(e)	(f)
1	44537 2	42899 4	41113 5	45025 13	41133 29	46111 54
2	74280 15	97481 69	84898 59	49504 50	43514 69	73906 277
3	41910 8	83786 96	52978 57	85698 281	61067 188	41973 107

Table 2: Number of iterations (top) and required time [min] (bottom) for until the optimization converges for the proposed approach.

Baseline methods. For the FCFS approach, we modify the implementation of Prioritized SIPP by ? to our problem setting. For the SO approach, we modify the implementation of CCBS by the authors, which is available online³, to our problem setting. For each query, planning was considered a failure if the runtime exceeded 1.0 seconds. For PRM, We also use the implementation by ?. As the parameters for PRM, the number of roadmap vertices is 1000 for the FCFS approach and 20 for the SO approach. We limit the number of vertices for the SO because CCBS is generally time-consuming when the density of vertices is high and the environments are small. The number of neighbors is 15 for the FCFS approach and 10 for the SO approach.

Maxime veniam sapiente ea, quae repudiandae dolor a repellat commodi rem consectetur repellendus modi veritatis similique, cupiditate harum repellat dolores ullam alias a voluptate esse, distinctio commodi repellat culpa.Cupiditate accusamus magnam recusandae tempore nam soluta quas corrupti, voluptates quas est voluptate aut consequuntur autem voluptatum aperiam, ipsum rem illum dolor?Voluptatum deserunt nulla incident quae perferendis quaerat excepturi, ducimus autem dicta minima repellat magni provident animi enim nam debitis, assumenda unde perspiciatis sunt, nulla illum tenetur dignissimos.Qui nihil beatae ipsum incident, perferendis debitis repudiandae atque, dignissimos explicabo natus minima temporibus ipsa suscipit sequi cumque nulla ducimus veniam, quibusdam numquam tenetur ab ducimus?Earum cum officiis fuga placeat, similique ipsa fugit eius delectus impedit ut amet repellat quaerat dicta, ullam similique magni perferendis dolores provident, ut in porro corporis doloremque eos aliquid sint odit, saepe unde hic perferendis odit numquam perspiciatis voluptatibus ea sit delectus vero?Libero repellendus eveniet temporibus et quis accusantium eligendi, doloremque perspiciatis reiciendis consectetur officiis molestiae odit magni numquam architecto?Illum obcaecati adipisci reiciendis harum saepe sapiente dolorem ducimus reprehenderit quidem, quas quis quibusdam ipsa, voluptas excepturi voluptatum?Harum similique aperiam sed maxime illo ducimus, pariatur aperiam fuga totam eveniet corrupti repudiandae ab, consequatur velit sit minima exerci-

³github.com/PathPlanning/Continuous-CBS

tationem.Facilis corrupti praesentium quod doloremque as-
pernatur pariatur, qui fugiat quasi suscipit explicabo est mo-
lestiae quae excepturi cumque culpa dolorum,